RAG-Based Customer Support Assistant

HIGH LEVEL DESIGN (HLD)

➢ Objective

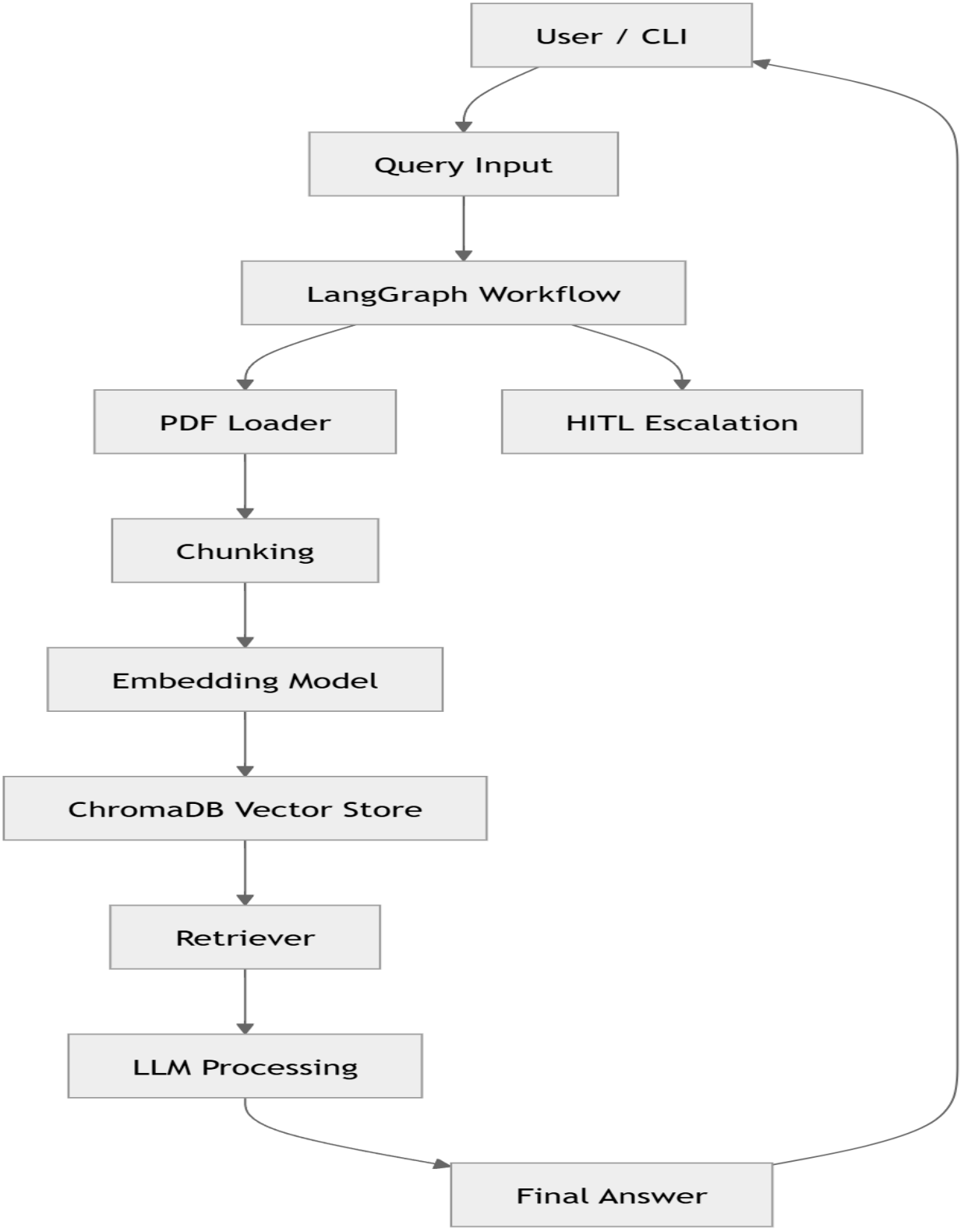
To design and implement a Retrieval-Augmented Generation (RAG) based Customer Support Assistant that retrieves relevant information from PDF documents and generates accurate, contextual responses using AI and workflow orchestration

➢ System Overview

- **Problem Definition**
  Traditional customer support systems are slow and cannot efficiently extract answers from large documents. Users need a system that can understand queries and provide accurate answers directly from knowledge base documents.

- **Scope of the System**
  The system is designed to:
  1. Process PDF documents
  2. Convert text into embeddings
  3. Store and retrieve information using vector database
  4. Generate intelligent responses using LLM
  5. Control workflow using LangGraph
  6. Support structured query processing for customer support use case

➢ Architecture Diagram



➢ Component Description

- **Document Loader**

Loads PDF files and extracts raw text content.

- **Chunking Strategy**

Splits large documents into smaller chunks for better retrieval accuracy.

- **Embedding Model**

Converts text into vector representations using Sentence Transformers.

- **Vector Store**

Stores embeddings in ChromaDB for fast similarity search.

- **Retriever**

Fetches relevant document chunks based on user query similarity.

- **LLM (Language Model)**

Generates final response using retrieved context.

- **Graph Workflow Engine**

LangGraph manages step-by-step execution of retrieval and generation flow.

- **Routing Layer**

Decides flow of execution based on query type and retrieved confidence.

➢ Data Flow

- **How data moves from PDF → Answer**

  Step 1: User Query Input
  The user enters a query through the CLI or Web interface. The query is sent to the system for processing.

  Step 2: Workflow Initialization
  The query is passed to the LangGraph workflow engine, which manages and controls the execution flow of the system.

  Step 3: PDF Processing (Ingestion Phase)
  The system processes the PDF document by extracting raw text and splitting it into smaller chunks for efficient handling.

  Step 4: Embedding Generation
  Each text chunk is converted into vector embeddings using an embedding model (Sentence Transformers), which represents semantic meaning.

  Step 5: Vector Storage
  The generated embeddings are stored in a vector database (ChromaDB) to enable fast similarity-based search.

  Step 6: Query Embedding
  The user query is also converted into an embedding vector using the same embedding model.

Step 7: Retrieval Process
The system performs similarity search in ChromaDB to retrieve the most relevant text chunks related to the query.

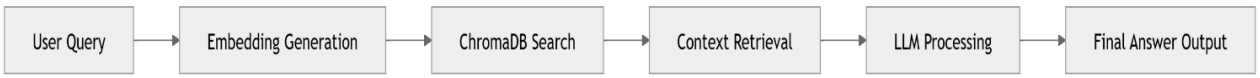
Step 8: Context Processing with LLM
The retrieved chunks are passed to the Large Language Model (LLM), which understands the context and prepares a response**.**

Step 9: Response Generation
The LLM generates a final, context-aware answer based on the retrieved information.

Step 10: Output Delivery
The final response is returned to the user through the interface.

Step 11: HITL (Optional Escalation)
If the system cannot find relevant information or confidence is low, the query is escalated to Human-in-the-Loop (HITL) or a fallback response is provided.

- **Query Lifecycle**



➢ Technology choices

- **Why ChromaDB?**
    1. Lightweight and fast vector database
    2. Easy integration with LangChain
    3. Efficient similarity search

- **Why LangGraph?**
    1. Provides structured workflow control
    2. Supports multi-step decision pipelines
    3. Enables routing and HITL integration

- **LLM Choice**
    1. Sentence Transformers / HuggingFace models
    2. Used for embeddings and response generation

- **Additional Tools**
    1. Python: Backend implementation
    2. LangChain: Orchestration framework
    3. PyPDF: PDF text extraction

➢ Scalability Considerations

- **Handling Large Documents**
  1. Chunking reduces processing load and improves manageability.
  2. Only relevant chunks are retrieved instead of processing full documents.

- **Increasing Query Load**
  1. Vector database (ChromaDB) enables fast similarity search even under high load.
  2. Stateless architecture allows better scalability for multiple users.

- **Latency Concerns**
  1. Precomputed embeddings reduce response time during query processing.
  2. Retrieval step reduces LLM input size, leading to faster response generation.

.